

# **Caso: Geração de AD com a ferramenta scikit-learn (modelo caixa branca)**

**Aula 03 – Inteligência Computacional**

Eduardo Corrêa Gonçalves

21/08/2026

# Sumário

## Introdução

- O que é scikit-learn?

## Treinamento de um classificador: passo a passo

- Importação da base de dados de treinamento

- Separação das partes  $X$  (*features*) e  $Y$  (classe)

- Pré-processamento da parte  $X$

- Treinamento do modelo

## Entendendo o modelo gerado

## Classificação de novos objetos: passo a passo

- Importação da base de dados com os novos objetos

- Pré-processamento

- Classificação

## Bônus: visualização gráfica da AD na scikit

# Introdução (1/2)

## O que é scikit-learn ?

A scikit-learn (<https://scikit-learn.org/>) é uma popular ferramenta livre de aprendizado de máquina (AM) desenvolvida em Python.

Oferece implementações dos principais algoritmos de AM:

- árvore de decisão;
- modelos lineares;
- rede neural;
- etc.

Oferece ainda funcionalidades para o pré-processamento de dados.

Também é conhecida como **sklearn**

# Introdução (2/2)

## O que é scikit-learn ?

A ferramenta é bem completa! Oferece muitas coisas!

Neste curso, vamos apresentar algumas de suas funcionalidades **de maneira gradual**, especialmente nas aulas com os estudos de caso.

Começaremos hoje examinando os passos básicos para treinar e utilizar um modelo de classificação.

# Treinamento do Classificador: passo a passo (1/12)

## Os 4 passos para treinar um classificador na scikit-learn

1. Importação da base de dados de treinamento.
2. Separação das partes X (*features*) e Y (classe).
3. Pré-processamento da parte X.
4. Treinamento do modelo.

Não é um esquema fixo! Dependendo do problema, você pode alterar esse fluxo.

### **Exemplos:**

Muitas vezes, a etapa 3 precisará ser dividida em várias subetapas (quando é necessário aplicar várias técnicas de pré-processamento) .

Outras vezes, ela pode nem ser necessária.

Algumas pessoas invertem a ordem de execução das etapas 2 e 3.

Outras separam X e Y (etapa 2) apenas no momento do treinamento (etapa 4)

# Treinamento do Classificador: passo a passo (2/12)

## Os 4 passos para treinar um classificador na scikit-learn

1. Importação da base de dados de treinamento.
2. Separação das partes X (*features*) e Y (classe).
3. Pré-processamento da parte X.
4. Treinamento do modelo.

Vamos demonstrar cada etapa utilizando o exemplo similar ao da aula passada:

Árvore de decisão treinada com os dados de uma pesquisa hipotética com pessoas analfabetas



| idade | uf | sexo | trab_domestic<br>o<br>(classe) |
|-------|----|------|--------------------------------|
| 40    | MG | M    | Não                            |
| 62    | SP | F    | Sim                            |
| 65    | MG | F    | Sim                            |
| 55    | SP | M    | Não                            |
| 41    | SP | F    | Não                            |
| 61    | MG | F    | Sim                            |
| 49    | SP | M    | Não                            |
| 65    | MG | M    | Não                            |
| 45    | MG | F    | Sim                            |
| 65    | SP | F    | Sim                            |
| 42    | MG | F    | Não                            |

# Treinamento do Classificador: passo a passo (3/12)

## PASSO 1: importação da base (BD) de treinamento

No cenário mais típico, a base de treino estará estruturada em um **arquivo CSV**.

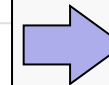
Trata-se do formato mais utilizado p/ armazenar BDs no formato tabular (dados dispostos em linhas e colunas).

Cada linha corresponde aos dados de um objeto (nesse exemplo, dados de uma pessoa)

E as colunas são separadas por vírgula.

| idade | uf | sexo | trab_domestico |
|-------|----|------|----------------|
| 40    | MG | M    | Não            |
| 62    | SP | F    | Sim            |
| 65    | MG | F    | Sim            |
| 55    | SP | M    | Não            |
| 41    | SP | F    | Não            |
| 61    | MG | F    | Sim            |
| 49    | SP | M    | Não            |
| 65    | MG | M    | Não            |
| 45    | MG | F    | Sim            |
| 65    | SP | F    | Sim            |
| 42    | MG | F    | Não            |

Conjunto de dados representado como tabela



```
pme.csv × + - □
Arquivo Editar Exibi
idade,uf,sexo,trab_domestico
40,MG,M,Não
62,SP,F,Sim
65,MG,F,Sim
55,SP,M,Não
41,SP,F,Não
61,MG,F,Sim
49,SP,M,Não
65,MG,M,Não
45,MG,F,Sim
65,SP,F,Sim
42,MG,F,Não
Ln 13, Col 1 161 caracteres 100% Winc
```

Mesmo conjunto de dados representado no formato CSV

# Treinamento do Classificador: passo a passo (4/12)

## PASSO 1: importação da base (BD) de treinamento

O arquivo CSV será importado para um **DataFrame** pandas (código abaixo).

A função **read\_csv** é a que importa (carrega) o conteúdo do arquivo CSV para um DataFrame, nesse exemplo **chamado** de “pme”

*Detalhes serão apresentados nas aulas sobre DataFrames e pré-processamento*

```
import pandas as pd

# -----
# Importa os dados de treinamento para um DataFrame
# -----
arq = 'https://raw.githubusercontent.com/edubd/bd/refs/heads/main/pme.csv'
pme = pd.read_csv(arq)

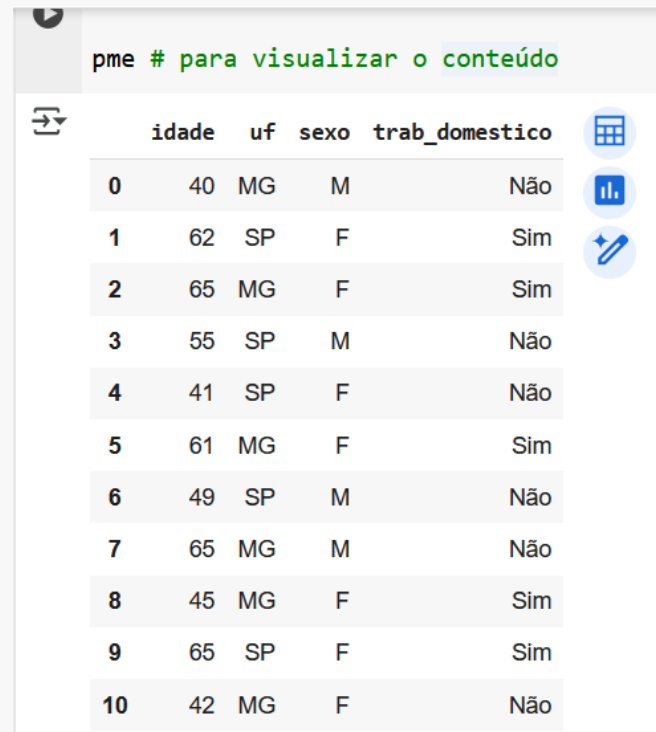
pme # para visualizar o conteúdo
```



# Treinamento do Classificador: passo a passo (5/12)

## Resultado ao final do PASSO 1

BD de treinamento terá sido importado p/ DataFrame



```
pme # para visualizar o conteúdo
```

|    | idade | uf | sexo | trab_domestico |
|----|-------|----|------|----------------|
| 0  | 40    | MG | M    | Não            |
| 1  | 62    | SP | F    | Sim            |
| 2  | 65    | MG | F    | Sim            |
| 3  | 55    | SP | M    | Não            |
| 4  | 41    | SP | F    | Não            |
| 5  | 61    | MG | F    | Sim            |
| 6  | 49    | SP | M    | Não            |
| 7  | 65    | MG | M    | Não            |
| 8  | 45    | MG | F    | Sim            |
| 9  | 65    | SP | F    | Sim            |
| 10 | 42    | MG | F    | Não            |

# Treinamento do Classificador: passo a passo (6/12)

## PASSO 2: separação das partes X (*features*) e Y (classe)

No momento de treinar o modelo a scikit exige que seja indicado quem é o conjunto de *features* X e quem é a classe Y. Então, para facilitar, fazemos a separação

É resolvida com a operação de **fatiamiento de DataFrames** (*mais um tema a ser detalhado nas aulas sobre preprocessamento*).

Veja que chamei X de “**X\_original**”... É porque na próxima etapa ela será pré-processado e só depois disso vai virar o X “de verdade” (ou X “final”)

```
# -----  
# separa a base de treinamento em X e Y  
# -----  
  
X_original = pme.iloc[:, [0, 1, 2]] # fatia c/ todas as linhas e as  
                                   # cols 0 (idade), 1 (uf) e 2 (sexo)  
  
Y = pme.iloc[:, 3] # fatia c/ todas as linhas e  
                   # a col. 3 (trab_domestico)
```

# Treinamento do Classificador: passo a passo (7/12)

## Resultado ao final do PASSO 2

X (*features*) e Y (classe) terão sido separados.

X ainda estará no formato original e precisará ser pré-processado antes do treinamento do modelo de classificação

```
print('- X (original):'); print(X_original)
print()
print('- Y:'); print(Y)
```

|    | idade | uf | sexo |
|----|-------|----|------|
| 0  | 40    | MG | M    |
| 1  | 62    | SP | F    |
| 2  | 65    | MG | F    |
| 3  | 55    | SP | M    |
| 4  | 41    | SP | F    |
| 5  | 61    | MG | F    |
| 6  | 49    | SP | M    |
| 7  | 65    | MG | M    |
| 8  | 45    | MG | F    |
| 9  | 65    | SP | F    |
| 10 | 42    | MG | F    |

| 0  | Não |
|----|-----|
| 1  | Sim |
| 2  | Sim |
| 3  | Não |
| 4  | Não |
| 5  | Sim |
| 6  | Não |
| 7  | Não |
| 8  | Sim |
| 9  | Sim |
| 10 | Não |

Name: trab\_domestico, dtype: object

# Treinamento do Classificador: passo a passo (8/12)

## PASSO 3: pré-processamento da parte X

Costuma ser a parte mais trabalhosa em qualquer projeto real de aprendizado de máquina e inteligência computacional.

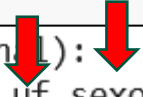
Quase nunca as *features* já vem originalmente em um formato que pode ser diretamente processado pelo algoritmo. 😞

No nosso exemplo específico:

A sklearn aceita apenas atributos numéricos na parte X.

Veja que no nosso X, “idade” é numérico, porém os atributos “uf” e “sexo” são categóricos!!

*E agora ???*



| - X (original): |       |    |      |
|-----------------|-------|----|------|
|                 | idade | uf | sexo |
| 0               | 40    | MG | M    |
| 1               | 62    | SP | F    |
| 2               | 65    | MG | F    |
| 3               | 55    | SP | M    |
| 4               | 41    | SP | F    |
| 5               | 61    | MG | F    |
| 6               | 49    | SP | M    |
| 7               | 65    | MG | M    |
| 8               | 45    | MG | F    |
| 9               | 65    | SP | F    |
| 10              | 42    | MG | F    |

# Treinamento do Classificador: passo a passo (9/12)

## PASSO 3: pré-processamento da parte X

A sklearn oferece diversos métodos para conversão de atributos categóricos em numéricos.

No nosso exemplo, vamos utilizar a técnica **Ordinal Encoder**, que converte a  $n$  categorias (string) para  $n$  números consecutivos: 0, 1, 2 etc.

*(de forma nada surpreendente... detalhes nas aulas sobre pré-processamento).*

```
# (1). importa o pacote com a técnica OrdinalEncoder
from sklearn.preprocessing import OrdinalEncoder

X = X_original.copy() # clona o X original

# (2). faz a conversão dos atributos uf e sexo p/ códigos inteiros
enc = OrdinalEncoder()

X[["uf", "sexo"]] = enc.fit_transform(X[["uf", "sexo"]])
```

# Treinamento do Classificador: passo a passo (10/12)

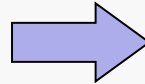
## Resultado ao final do PASSO 3

X\_original terá sido pré-processado.

Ele estará no formato exigido pelo algoritmo de classificação da scikit.

- X (original):

|    | idade | uf | sexo |
|----|-------|----|------|
| 0  | 40    | MG | M    |
| 1  | 62    | SP | F    |
| 2  | 65    | MG | F    |
| 3  | 55    | SP | M    |
| 4  | 41    | SP | F    |
| 5  | 61    | MG | F    |
| 6  | 49    | SP | M    |
| 7  | 65    | MG | M    |
| 8  | 45    | MG | F    |
| 9  | 65    | SP | F    |
| 10 | 42    | MG | F    |



- X (pré-processado):

|    | idade | uf  | sexo |
|----|-------|-----|------|
| 0  | 40    | 0.0 | 1.0  |
| 1  | 62    | 1.0 | 0.0  |
| 2  | 65    | 0.0 | 0.0  |
| 3  | 55    | 1.0 | 1.0  |
| 4  | 41    | 1.0 | 0.0  |
| 5  | 61    | 0.0 | 0.0  |
| 6  | 49    | 1.0 | 1.0  |
| 7  | 65    | 0.0 | 1.0  |
| 8  | 45    | 0.0 | 0.0  |
| 9  | 65    | 1.0 | 0.0  |
| 10 | 42    | 0.0 | 0.0  |

### Códigos:

**uf**  
MG = 0.0  
SP = 1.0

**sexo**  
F = 0.0  
M = 1.0

# Treinamento do Classificador: passo a passo (11/12)

## PASSO 4: treinamento do modelo de classificação

(1). **importar o pacote** com o algoritmo que você quer usar, nesse caso CART (pacote DecisionTreeClassifier da scikit)

(2). **Instanciar o classificador**, indicando os parâmetros que você deseja.

Instanciar é como carregar o algoritmo em memória para posterior utilização.

(3). Usar o método (função) **fit**, para fazer o **treinamento do classificador**, indicando X e Y.

```
# (1). importa a classe DecisionTreeClassifier do módulo
sklearn.tree
from sklearn.tree import DecisionTreeClassifier

# (2). instancia (cria um objeto) DecisionTreeClassifier,
# usando os parâmetros:
#     índice de Gini como medida de impureza (criterion)
#     e 'random_state = 0' p/ gerar sempre a mesma árvore
modelo = DecisionTreeClassifier(criterion='gini', random_state=0)

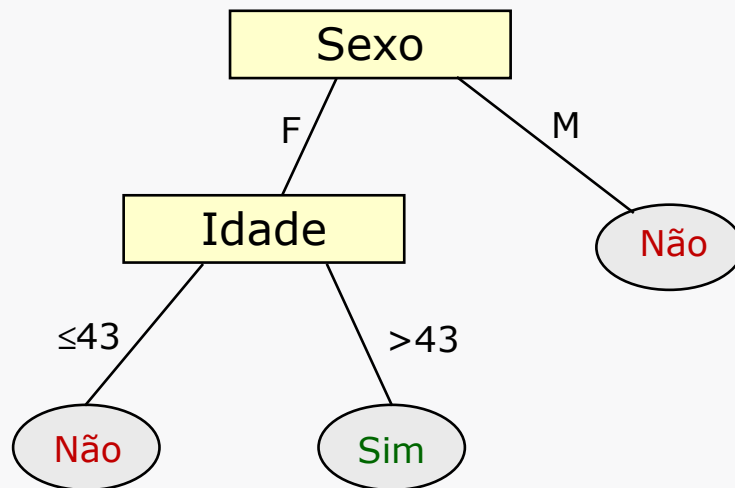
# (3). treina o modelo de AD CART com o método fit(),
#     indicando X e Y
modelo.fit(X, Y)
```

# Treinamento do Classificador: passo a passo (12/12)

## Resultado ao final do PASSO 4

Um modelo de classificação AD CART treinado em memória 😊

Pena que, ao imprimir a AD gerada, notamos que a scikit não usa uma representação tão clara como poderia ser...



Essa é a AD que o algoritmo aprendeu

```

| --- sexo <= 0.50
|   | --- idade <= 43.50
|   |   | --- class: Não
|   |   | --- idade > 43.50
|   |   | --- class: Sim
| --- sexo > 0.50
|   | --- class: Não
  
```

Esse é o jeito que a sklearn te mostra por padrão



# Entendendo o modelo gerado (1/6)

## Regras

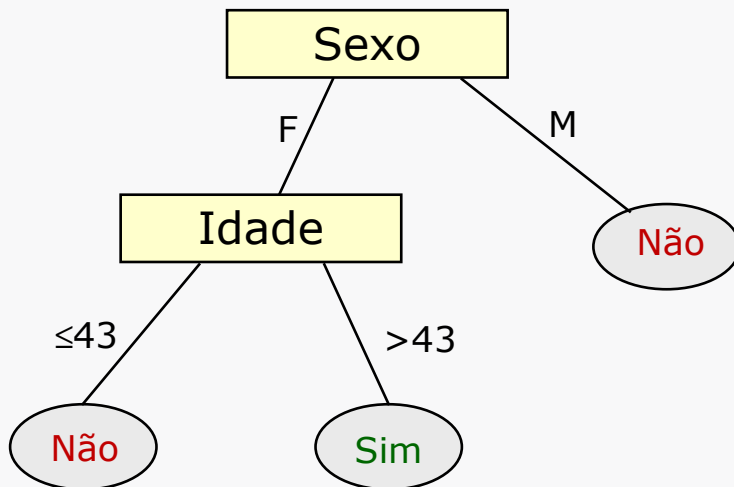
Olha a representação no lado esquerdo (a bonita...)

Veja que a AD possui **3 nós folha**, logo há **3 regras** de classificação. São elas:

(Sexo = "F") & (idade  $\leq$  43)  $\Rightarrow$  "Não"

(Sexo = "F") & (idade > 43)  $\Rightarrow$  "Sim"

(Sexo = "M")  $\Rightarrow$  "Não"



```
| --- sexo <= 0.50
|   | --- idade <= 43.50
|   |   | --- class: Não
|   |   | --- idade > 43.50
|   |   | --- class: Sim
|   | --- sexo > 0.50
|   | --- class: Não
```

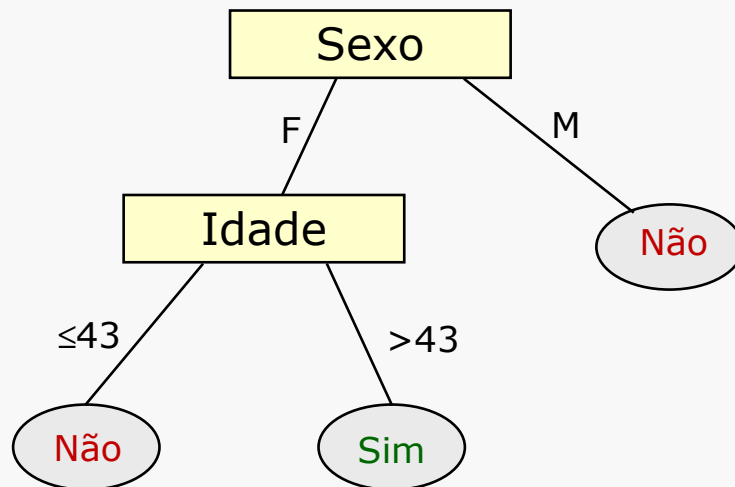
# Entendendo o modelo gerado (2/6)

## Nós folha

Mas onde estão essas regras na representação da sklearn ?

Primeiro vamos identificar os **nós folha**

São os nós escritos como **class: Não** e **class: Sim**



```
| --- sexo <= 0.50
|   | --- idade <= 43.50
|   |   | --- class: Não
|   |   | --- idade > 43.50
|   |   |   | --- class: Sim
|   |   |   | --- class: Não
|   | --- sexo > 0.50
|   |   | --- class: Não
```

Blue dashed arrows point from the text 'nós folha' to the leaf nodes in the decision tree and from the text 'class: Não' and 'class: Sim' to the corresponding lines in the sklearn representation.

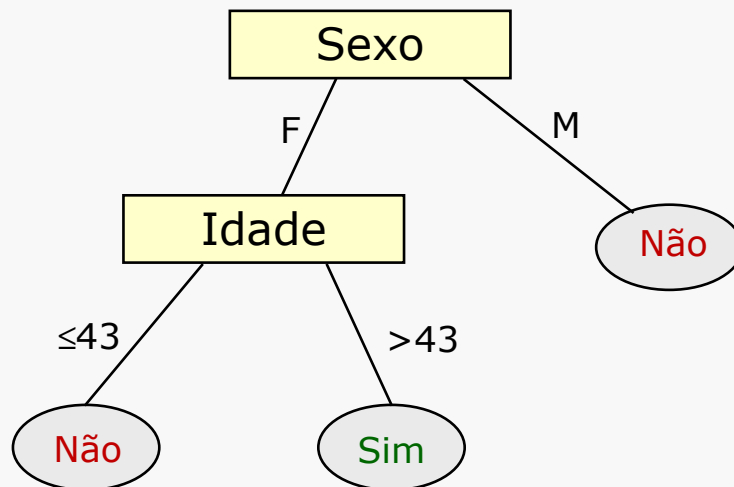
# Entendendo o modelo gerado (3/6)

## Nó raiz

Agora vamos achar o **nó raiz**

O **nó raiz** é o mais à esquerda, “**sexo**”

Para identificar cada regra de classificação, basta agora percorrer o diagrama da esquerda para direita, de cima para baixo



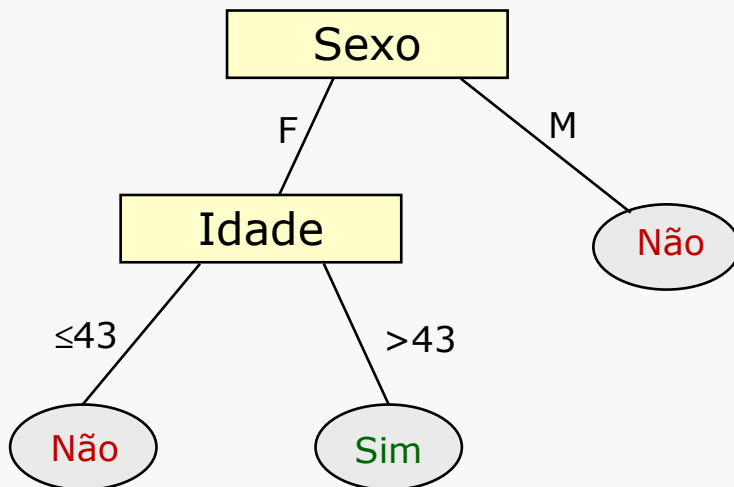
```
| --- sexo <= 0.50
| | --- idade <= 43.50
| | | --- class: Não
| | | --- idade > 43.50
| | | --- class: Sim
| --- sexo > 0.50
| | --- class: Não
```

# Entendendo o modelo gerado (4/6)

## Regras

Pronto ! Para identificar cada regra de classificação, basta agora percorrer o diagrama da **esquerda para direita**, de **cima para baixo**.

Aqui está destacada uma das regras:



```

| --- sexo <= 0.50
|   | --- idade <= 43.50
|   |   | --- class: Não
|   |   | --- idade > 43.50
|   |   | --- class: Sim
| --- sexo > 0.50
|   | --- class: Não
  
```

# Entendendo o modelo gerado (5/6)

regra: (sexo <= 0.50) & (idade <= 43.5)  $\Rightarrow$  class: “Não”

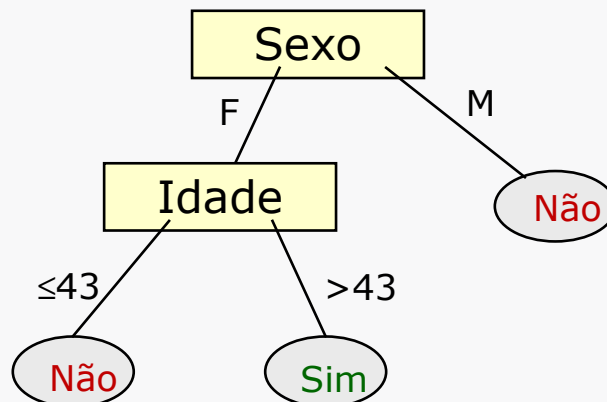
<= significa apenas o sinal  $\leq$

Mas o que significa sexo <= 0.50 ????

**Explicação:** no pré-processamento, o atributo “sexo” precisou ser codificado, lembra? “F” virou 0.0 e “M” virou 1.0

O CART sempre escolhe como ponto de corte o ponto médio entre 2 valores do atributo. O **ponto médio** entre 0.0 e 1.0 é 0.5.

**Sendo assim:** sexo <= 0.5 é a mesma coisa que sexo = 0.0 que, no fim, é sexo = ‘F’ (!!!!!!!!)



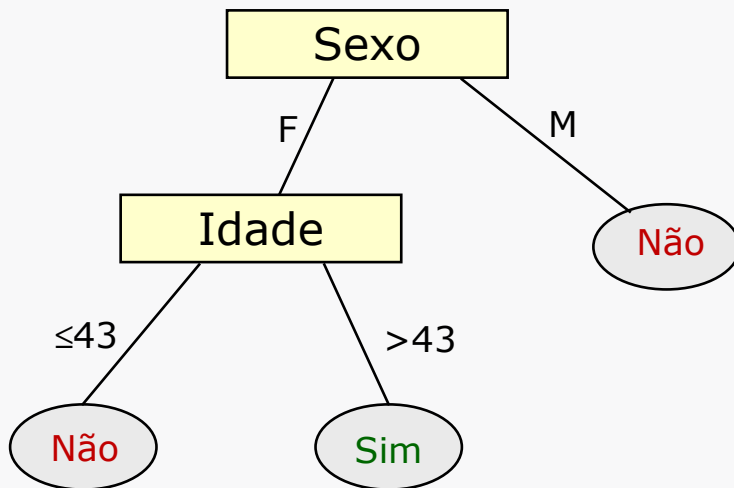
```
| --- sexo <= 0.50
| --- idade <= 43.50
| | --- class: Não
| --- idade > 43.50
| | --- class: Sim
| --- sexo > 0.50
| | --- class: Não
```

## Entendendo o modelo gerado (6/6)

Desta forma a regra no jeito feio:  $(\text{sexo} \leq 0.50) \ \& \ (\text{idade} \leq 43.5) \Rightarrow \text{class: "Não"}$

Corresponde à regra no jeito bonito:  $(\text{Sexo} = \text{"F"}) \ \& \ (\text{idade} \leq 43) \Rightarrow \text{"Não"}$

**EXERCÍCIO**: identifique as demais regras na representação da AD da sklearn.



```
| --- sexo <= 0.50
|   | --- idade <= 43.50
|   |   | --- class: Não
|   |   | --- idade > 43.50
|   |   | --- class: Sim
|   | --- sexo > 0.50
|   | --- class: Não
```

# Classificação de novos objetos: passo a passo (1/7)

## Os 3 passos para utilizar um classificador treinado na scikit-learn

1. Definição ou importação dos novos objetos
2. Pré-processamento dos dados dos novos objetos.
3. Utilização do modelo treinado para classifica-los

De novo, não é um esquema fixo!

Dependendo do problema, você pode alterar esse fluxo.

# Classificação de novos objetos: passo a passo (2/7)

## PASSO 1: define ou importa os novos objetos

Em nosso exemplo, defini manualmente um único objeto novo em um DataFrame.

**Características:** 62 anos, moradora de SP, sexo feminino.

Também é possível ler um BD com vários objetos novos (na sklearn, posso aplicar o modelo para classificar vários objetos de uma vez)

```
# define o novo objeto  
novoObj = pd.DataFrame({"idade": [62], "uf": ["SP"], "sexo": ["F"]})
```



# Classificação de novos objetos: passo a passo (3/7)

## Resultado ao final do PASSO 1

novo(s) objeto(s) terá(ão) sido importado(s) para um DataFrame

```
print('- Novo objeto:');  
novoObj
```



- Novo objeto:

|  | idade | uf | sexo |
|--|-------|----|------|
|--|-------|----|------|

|   |    |    |   |
|---|----|----|---|
| 0 | 62 | SP | F |
|---|----|----|---|



# Classificação de novos objetos: passo a passo (4/7)

## PASSO 2: pré-processamento dos novos objetos

Todo **pré-processamento** feito com os dados de treinamento precisará ser **reproduzido com os novos objetos**.

**Motivo:** é preciso os novos objetos estejam com os **dados** no **mesmo formato usado no treinamento**, pois esse é o formato que o modelo treinado conhece.

**No nosso exemplo:** precisaremos converter os atributos “uf” e “**sexo**” para códigos numéricos, usando o mesmo conversor OrdinalEncoder do treinamento

Basta usar o método **transform** do conversor **enc** (conversor criado no treinamento)

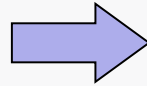
```
# codifica os dados do objeto de teste usando o método transform()
# do codificador OrdinalEncoder "enc" criado no treinamento
novoObj[["uf", "sexo"]] = enc.transform(novoObj[["uf", "sexo"]])
```

# Classificação de novos objetos: passo a passo (5/7)

## Resultado ao final do PASSO 2

O novo objeto terá sido pré-processado adequadamente.  
Ele estará no formato exigido pelo modelo AD que foi treinado.

```
- Novo objeto (original):  
  idade  uf  sexo  
0      62  SP    F
```



```
- Novo objeto (pré-processado):  
  idade  uf  sexo  
0      62  1.0  0.0
```

### Códigos:

**uf**  
MG = 0.0  
SP = 1.0

**sexo**  
F = 0.0  
M = 1.0

# Classificação de novos objetos: passo a passo (6/7)

## PASSO 3: classificação

Consiste simplesmente em **aplicar o modelo** treinado (em nosso caso uma AD CART) **para classificar** o novo objeto.

Isso é feito com o método **predict** do modelo.

Ele vai retornar uma das classes possíveis (“Sim” ou “Não” em nosso exemplo)

**IMPORTANTE:** podemos fazer o predict de vários objetos novos de uma vez...

Nesse caso, seria retornado uma lista contendo a classe predita para todos os objetos

```
# aplica o modelo de AD treinado para classificar o novo objeto
# usando o método "predict"
classe_predita = modelo.predict(novoObj)

print('A classe do novo objeto é: ', classe_predita)
```



A classe do novo objeto é: ['Sim']

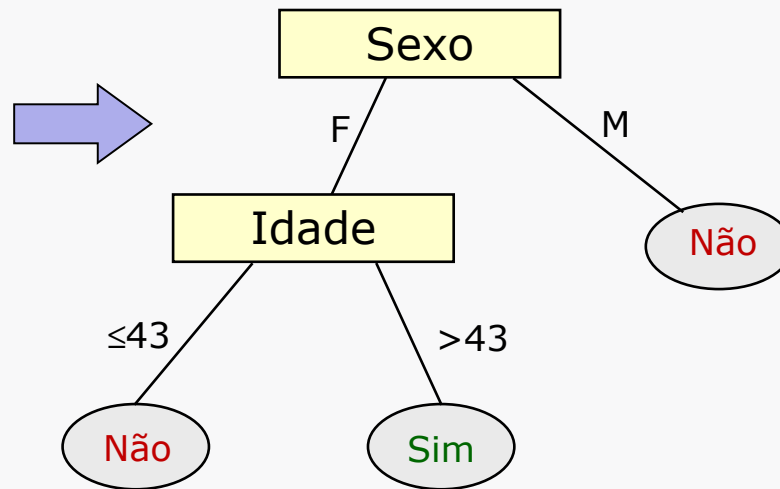
# Classificação de novos objetos: passo a passo (7/7)

## Resultado ao final do PASSO 3

Todos os novos objetos terão sido classificados

| idade | uf | sexo |
|-------|----|------|
| 62    | SP | F    |

**Novo objeto**  
(dados de  
uma pessoa)



**Classificador**  
(AD CART)

**Sim**

**Classe determinada  
pela AD**

# Link para código com o exemplo da aula

- **Caso – Classificação Caixa Branca com AD (exemplo dessa aula)**
  - [https://colab.research.google.com/drive/1f4UBr\\_o-mgTiD-fp7B7dAPhIxiWF4VL?usp=sharing](https://colab.research.google.com/drive/1f4UBr_o-mgTiD-fp7B7dAPhIxiWF4VL?usp=sharing)

# EXERCÍCIO

- Analise a AD abaixo gerada pela sklearn e responda as seguintes questões:
  - (a) Qual atributo está no nó raiz ?
  - (b) Escreva as regras de classificação representadas na AD (*considere que os atributos preditivos A, B e C são binários*)

```

|--- A <= 0.50
|   |--- B <= 0.50
|   |   |--- C <= 0.50
|   |   |   |--- class: Negativo
|   |   |--- C > 0.50
|   |   |--- class: Positivo
|   |--- B > 0.50
|   |--- class: Positivo
|--- A > 0.50
|   |--- class: Negativo

```

# Principais conceitos apresentados

- **scikit-learn:** ferramenta livre para o Python que oferece diversos algoritmos de aprendizado de máquina (AM) e pré-processamento. Também chamada de **scikit** e **sklearn**.
- **Passos para treinar um classificador na sklearn:** (i) importação da base de dados de treinamento; (ii) separação das partes X (*features*) e Y (classe); (iii) pré-processamento da parte X; (iv) Treinamento do modelo.
- **Pré-processamento:** processo que consiste em transformar os dados para que fiquem no formato exigido ou mais adequado para ser processado por um algoritmo.
- **Importação de algoritmo:** ação que consiste em importar o módulo da scikit que contém o algoritmo que você deseja utilizar em seu programa.
- **Instanciar classificador:** ação que consiste em carregar o algoritmo em memória para posterior utilização.
- **fit:** método (função) utilizado para treinar um classificador instanciado na sklearn. Recebe como entrada as partes X e Y da base de dados de treinamento.
- **Passos para classificar na sklearn:** (i) definição ou importação dos novos objetos; (ii) pré-processamento; (iii) classificação usando o modelo treinado.
- **Aplicar um modelo:** utilização de um modelo treinado para classificar um ou mais novos objetos.



# Bônus: visualização gráfica da AD na scikit (1/5)

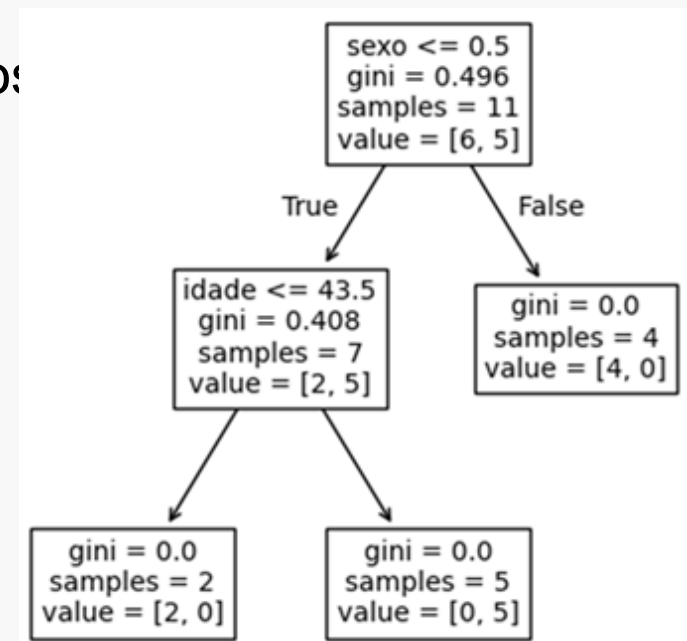
A scikit também permite “plotar” a AD como uma figura

Fica mais bonita que a versão texto, além de fornecer informações adicionais.

Coloquei o código no notebook da nos

```
--- sexo <= 0.50
|   |--- idade <= 43.50
|   |   |--- class: Não
|   |   |--- idade > 43.50
|   |   |--- class: Sim
|   |--- sexo > 0.50
|   |--- class: Não
```

Esse é o jeito que a scikit te mostra por padrão



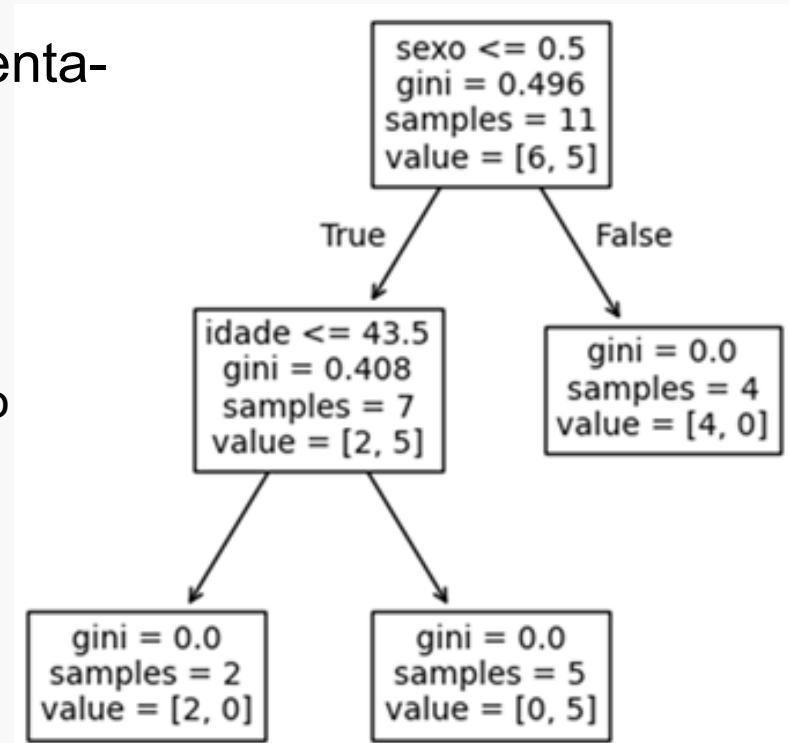
Essa é a versão como figura

# Bônus: visualização gráfica da AD na scikit (2/5)

## Características

Para cada nó (retângulo da figura), apresenta-se:

- O atributo e teste do particionamento
- O valor do índice de gini
- O total de objetos dentro do particionamento
- O total de objetos de cada classe.



# Bônus: visualização gráfica da AD na scikit (3/5)

## Exemplo 1 – nó raiz

Contém todos os dados do BD

atributo: **sexo**

teste que vai dividir os dados: **sexo  $\leq$  0.5**

(como vimos, significa **sexo = “F”**)

O caminho do resultado True (verdadeiro) fica sempre à esquerda e False (Falso) a direita

gini da partição: **0.496** (nesse caso, todo o BD)

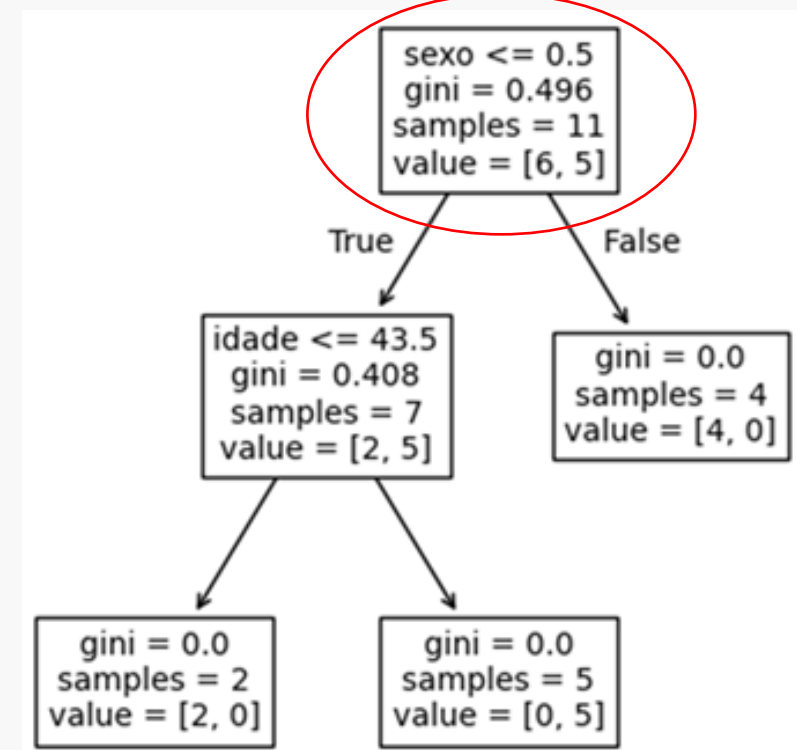
total de objetos: **11** (todo o BD)

total de objetos de cada classe:

**Não = 6 objetos**

**Sim = 5 objetos**

*Ele apresenta na ordem alfabética dos rótulos de classe, por isso sei que o valor 6 é do “Não” e o 5 é do “Sim”*



# Bônus: visualização gráfica da AD na scikit (4/5)

## Exemplo 2 – nó interno

Contém os dados definidos pela condição  
**sexo**  $\leq 0.5$ , definida pelo nó anterior

atributo: **idade**

teste que vai dividir os dados: **idade**  $\leq 43.5$

O caminho do resultado True (verdadeiro) fica sempre à esquerda e False (falso) à direita (mas ele só mostra explicitamente no nó raiz)

gini da partição: **0.408**

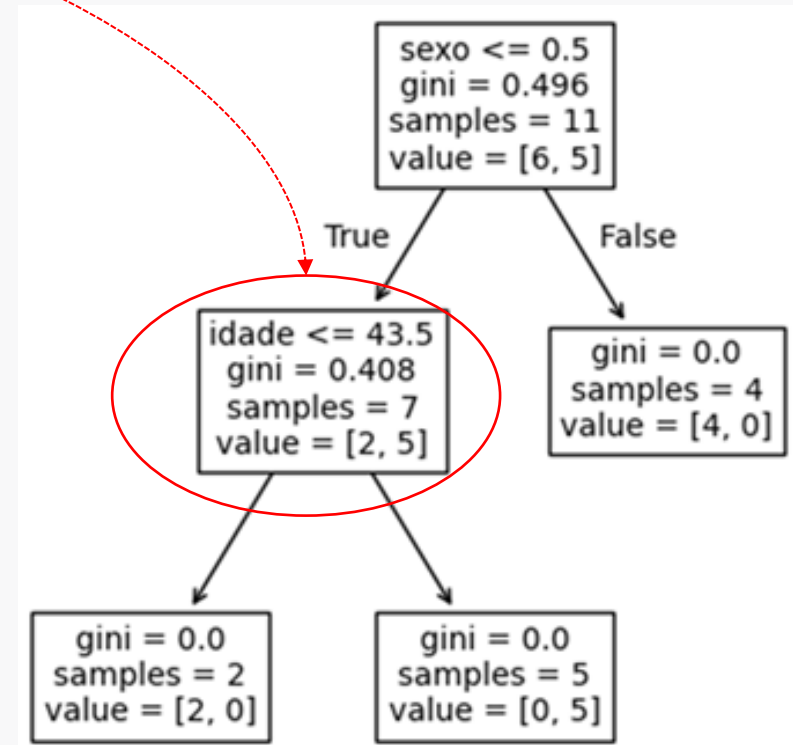
nesse caso, são os dados filtrados pela condição **sexo**  $\leq 0.5$  que define a partição

total de objetos: **7**

total de objetos de cada classe:

**Não** = 2 objetos

**Sim** = 5 objetos



# Bônus: visualização gráfica da AD na scikit (5/5)

## Exemplo 3 – nó folha

Contém os dados definidos pela regra  
(sexo  $\leq 0.5$ ) & (idade  $> 43.4$ ) = Sim , definida  
pelo caminho até esse nó.

gini da partição: 0.0

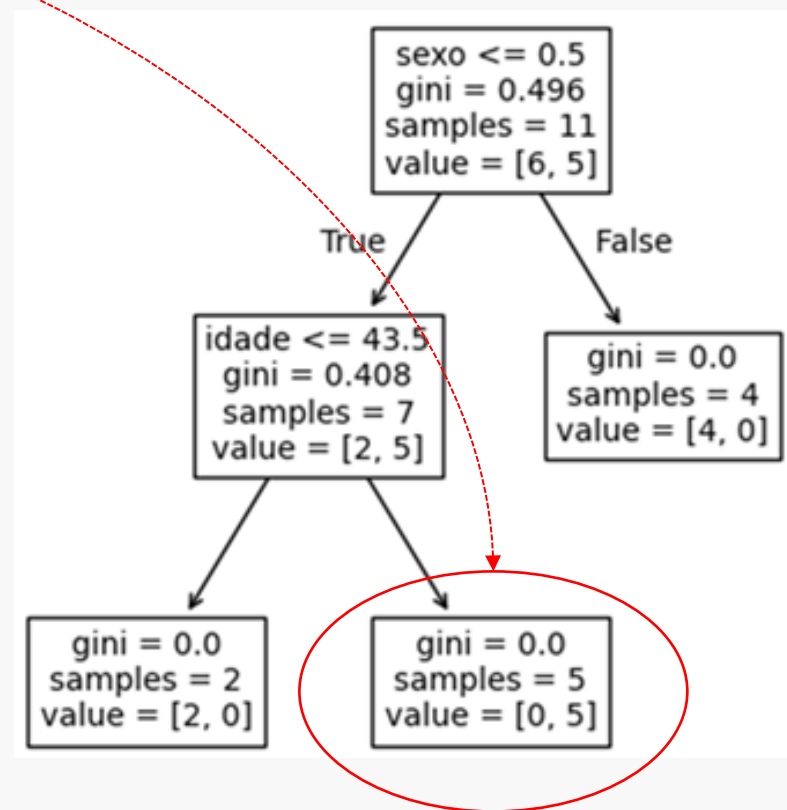
Partição totalmente pura

O total de objetos: 5

O total de objetos de cada classe:

Não = 0 objetos

Sim = 5 objetos



# Referências

HAN, J.; KAMBER, M. PEI, J. Classification: Basic Concepts. *In*: HAN, J.; KAMBER, M. PEI, J. **Data Mining: Concepts and Techniques**. 3. ed. Waltham: Morgan Kauffman, 2011. cap 8, p. 327 – 350.

Decision Trees. *In*: scikit-learn documentation. Disponível em < <https://scikit-learn.org/stable/modules/tree.html> >. Acesso em: 05 abr. 2025.